INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

OFFLINE CLASS RULES

TO BE FOLLOWED VERY STRICTLY

1. BE PRESENT IN THE CLASS ATLEAST 5 MINUTES BEFORE THE START OF THE CLASSES
2. STUDENTS WILL NOT BE ALLOWED TO ENTER THE CLASS AFTER 9.30AM AND 11.00AM
3. USE OF CELLPHONES & LAPTOPS STRICTLY PROHIBITED INSIDE THE CLASS
4. CELL PHONES & LAPTOPS SHOULD NOT BE KEPT ON THE DESK
5. CELLPHONES SHOULD BE KEPT EITHER IN YOUR POCKET OR BACKPACK/BAG YOU CARRY
6. SLEEPING IN CLASS IS TOTALLY PROHIBITED
7. BRING A NOTE BOOK FOR THIS COURSE AND MAKE NOTES IN THIS BOOK DURING THE CLASS
8. NO DRINKING & NO EATING IS ALLOWED IN CLASS (PLEASE NOTE THAT CLASSROOM IS NOT CANTEEN)

OFFLINE CLASS RULES

TO BE FOLLOWED VERY STRICTLY

- BE PRESENT IN THE CLASS ATLEAST 5 MINUTES BEFORE THE START OF THE CLASSES

- STUDENTS WILL NOT BE ALLOWED TO ENTER THE CLASS AFTER 9.30AM AND 11.00AM

SOC 2040
SYSTEM PROGRAMMING

WEEK 11 LECTURE 2
MEMORY HIERARCHY PART 3

CACHE PERFORMANCE METRICS

Cache Performance Metrics

- **Miss Rate**
  - Fraction of memory references not found in cache (misses / accesses)
    = 1 – h where **h=hit rate = Total # of Hits/ Total # of References**
  - Typical numbers (in percentages):
    - 3-10% for L1
    - can be quite small (e.g., < 1%) for L2, depending on size, etc.

- **Hit Time**
  - Time to deliver a line in the cache to the processor
    - includes time to determine whether the line is in the cache
  - Typical numbers:
    - 4 clock cycle for L1
    - 10 clock cycles for L2

- **Miss Penalty**
  - Additional time required because of a miss
    - typically 50-200 cycles for main memory (Trend: increasing!)

Cache Performance

- A better measure of memory hierarchy performance is the average memory access time:

Average memory access time

= Hit time + Miss rate x Miss penalty

where Hit time is the time to hit in the cache.

Multilevel Caches

- **Average memory access time for a two-level cache:**

- Using the subscripts L1 and L2 to refer, respectively, to a first-level and a second level cache, the original formula is

  Average memory access time=

  $\text{Hit time}_{L1} + \text{Miss rate}_{L1} \times \text{Miss penalty}_{L1}$

  where

  $\text{Miss penalty}_{L1} = \text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2}$

Average memory access time =

$\text{Hit time}_{L1} + \text{Miss rate}_{L1} \times (\text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2})$

Multilevel Caches

Given a Computer System having a processor with two levels of Caches and Main Memory with the following parameters:

L1 Cache : Hit Rate h_{L1}, Hit Time ht_{L1} (in clock cycles)

L2 Cache : Hit Rate h_{L2}, Hit Time ht_{L2} (in clock cycles)

Main Memory : Miss Penalty mp (in clock cycles)

Write the expression for computing the Average Time for accessing the Cache block

Average Access Time $T_{av} = ht_{L1} + (1 - h_{L1})(ht_{L2} + (1- h_{L2}) *mp)$

Given a Computer System having a processor with three levels of Caches and Main Memory with the following parameters:

L1 Cache : Hit Rate h_{L1}, Hit Time ht_{L1} (in clock cycles)

L2 Cache : Hit Rate h_{L2}, Hit Time ht_{L2} (in clock cycles)

L3 Cache : Hit Rate h_{L3}, Hit Time ht_{L3} (in clock cycles)

Main Memory : Miss Penalty mp (in clock cycles)

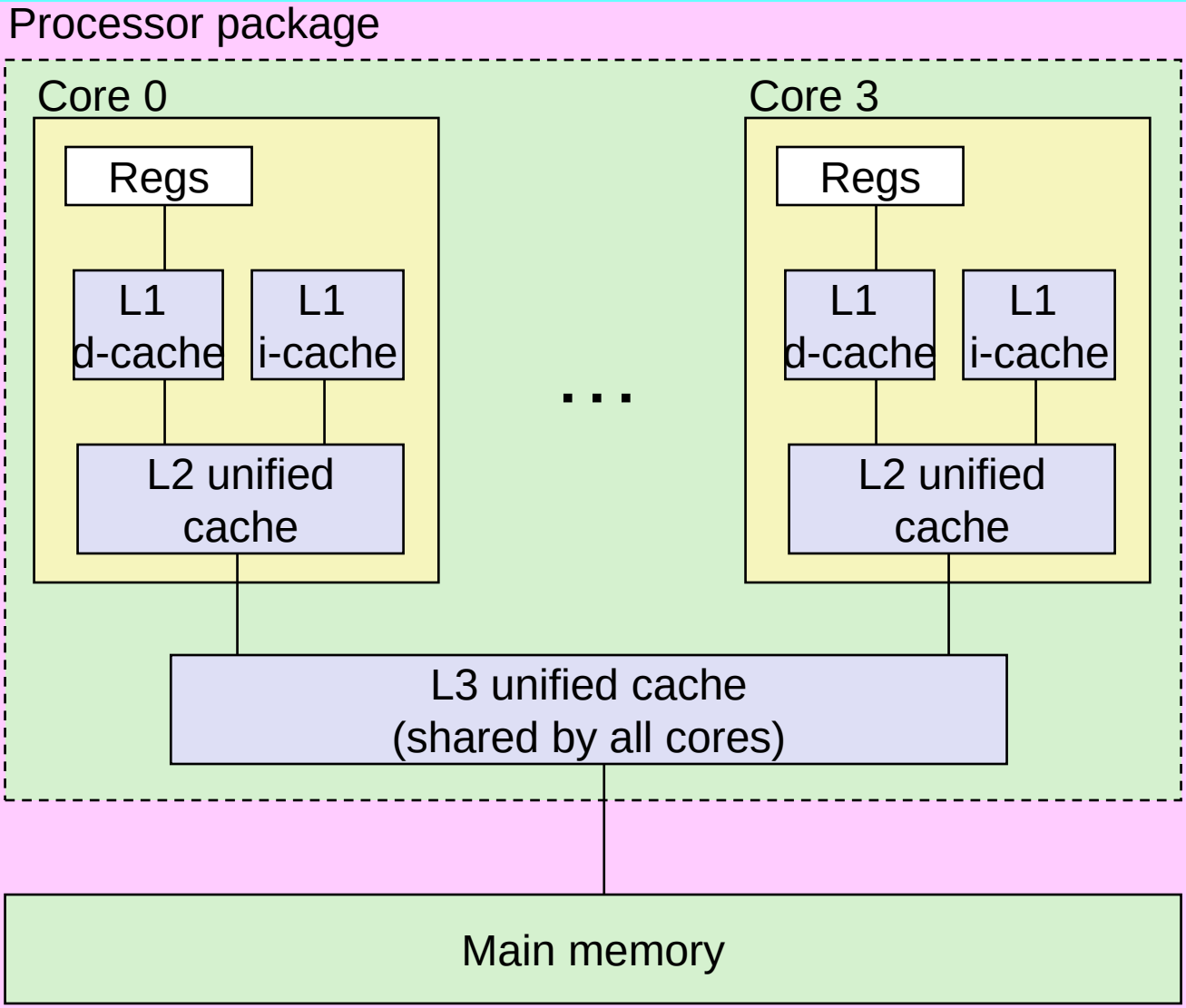
Write the expression for computing the Average Time for accessing the Cache block

Average Access Time $T_{av} = ht_{L1} + (1 - h_{L1})(ht_{L2} + (1- h_{L2})*(ht_{L3} + (1- h_{L3})*mp))$

General Caching Concepts: Types of Cache Misses

- **Cold (compulsory) miss**
  - Cold misses occur because the cache is empty.
- **Conflict miss**
  - Most caches limit blocks at level k+1 to a small subset (sometimes a singleton) of the block positions at level k.
    - E.g. Block i at level k+1 must be placed in block (i mod 4) at level k.
  - Conflict misses occur when the level k cache is large enough, but multiple data objects all map to the same level k block.
    - E.g. Referencing blocks 0, 8, 0, 8, 0, 8, ... would miss every time.
- **Capacity miss**
  - Occurs when the set of active cache blocks (working set) is larger than the cache.

Intel Core i7 Cache Hierarchy



L1 Split caches :
i-cache and d-cache:
32 KB, 8-way,
Access: 4 cycles

L2 unified cache:
256 KB, 8-way,
Access: 10 cycles

L3 unified cache:
8 MB, 16-way,
Access: 40-75
cycles

Block size: 64 bytes for all caches.

INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

25 April 2024

SOC 2040
SYSTEM PROGRAMMING

MEMORY HIERARCHY PART 3

PRINCIPLE OF LOCALITY

PRINCIPLE OF LOCALITY

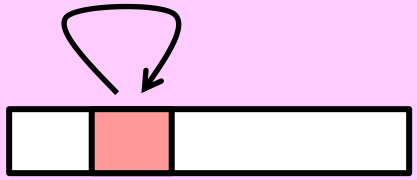
- **The key to bridging this CPU-Memory gap is a fundamental property of computer programs known as locality**
- **Well-written computer programs tend to exhibit good locality.**
- **Principle of Locality:**

✸ Programs tend to use data and instructions with addresses near or equal to those they have used recently

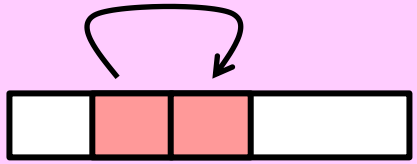
Principle of Locality

- **Two kinds of Locality**

- Temporal locality:
  - Recently referenced items are likely to be referenced again in the near future



- Spatial locality:
  - Items with nearby addresses tend to be referenced close together in time



PRINCIPLE OF LOCALITY

- **In a program with good temporal locality, a memory location that is referenced once is likely to be referenced again multiple times in the near future.**
- **In a program with good spatial locality, if a memory location is referenced once, then the program is likely to reference a nearby memory location close together in time.**
- **Programmers should understand the principle of locality because, in general, programs with good locality run faster than programs with poor locality.**

PRINCIPLE OF LOCALITY

- All levels of modern computer systems, from the hardware, to the operating system, to application programs, are designed to exploit locality.
- At the hardware level, the principle of locality allows computer designers to speedup main memory accesses by introducing small fast memories known as cache memories that hold blocks of the most recently referenced instructions and data items.
- At the operating system level, the principle of locality allows the system to use the main memory as a cache of the most recently referenced chunks of the virtual address space.
- Similarly, the operating system uses main memory to cache the most recently used disk blocks in the disk file system.
- The principle of locality also plays a crucial role in the design of application programs.
- For example, Web browsers exploit temporal locality by caching recently referenced documents on a local disk.
- High-volume Webservers hold recently requested documents in front-end disk caches that satisfy requests for these documents without requiring any intervention from the server.

Locality Example

Consider the simple function in Figure that sums the elements of a vector.
Q) Does this function have good locality?
To answer this question, look at the reference pattern for each variable

```
int sumvec (int v[N])
{ int i, sum = 0;
  for (i = 0; i < n; i++)
      sum += v[i];
   return sum; }
```

Address	0	4	8	12	16	20	24	28
Contents	v_0	v_1	v_2	v_3	v_4	v_5	v_6	v_7
Access order	1	2	3	4	5	6	7	8

- **Data references**
  - Array elements v[i] are referenced in succession one after the other (**stride-1 reference pattern**).
  - Variable `sum` is referenced once in each loop iteration.
- **Instruction references**
  - Instructions Referenced in sequence.
  - Cycling through loop repeatedly.

Spatial locality

Exhibits Temporal locality

Exhibits Spatial locality

ExhibitsTemporal locality

Since the function has either good spatial or temporal locality with respect to each variable in the loop body, we can conclude that the sumvec function enjoys good locality.

PRINCIPLE OF LOCALITY

Stride-k

- A function such as sumvec that visits each element of a vector sequentially is said to have a stride-1 reference pattern (with respect to the element size).

```
for (i = 0; i < n; i++)
        sum += v[i];
```

- Sometimes stride-1 reference patterns are referred as sequential reference patterns.
- Visiting every kth element of a contiguous vector is called a stride-k reference pattern.
- Stride-1 reference patterns are a common and important source of spatial locality in programs.
- In general, as the stride increases, the spatial locality decreases.
- Stride is also an important issue for programs that reference multidimensional arrays.

Qualitative Estimates of Locality

- Claim: Being able to look at code and get a qualitative sense of its locality is a key skill for a professional programmer.
- Question: Does this function have good locality with respect to array `a`?

```
int sum_array_rows(int a[M][N])
{
    int i, j, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

Address	0	4	8	12	16	20
Contents	a_{00}	a_{01}	a_{02}	a_{10}	a_{11}	a_{12}
Access order	1	2	3	4	5	6

M=2, N=3

- **Consider the sum_array_rows function in Figure that sums the elements of a two-dimensional array.**
- **The doubly nested loop reads the elements of the array in row-major order.**
- **That is, the inner loop reads the elements of the first row, then the second row, and soon.**
- **The sum_array_rows function enjoys good spatial locality because it references the array in the same row-major order that the array is stored.**
- **The result is a nice stride-1 reference pattern with excellent spatial locality.**

Locality Example

Question:

- **Does this function have good locality with respect to array a?**

```
int sum_array_cols(int a[M][N])
{
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Address	0	4	8	12	16	20
Contents	a_{00}	a_{01}	a_{02}	a_{10}	a_{11}	a_{12}
Access order	1	3	5	2	4	6

M=2, N=3

Seemingly trivial changes to a program can have a big impact on its locality.

For example, the sum_array_cols function in Figure computes the same result as the sum_array_rows function. The only difference is that we have interchanged the i and j loops.

What impact does interchanging the loops have on its locality?

The sum_array_cols function suffers from poor spatial locality because it scans the array column-wise instead of row-wise.

Since C arrays are laid out in memory row-wise, the result is a stride-N reference pattern, as shown in Figure.

Locality Example

Question:

- Can you permute the loops so that the function scans the 3-d array `a` with a stride-1 reference pattern (and thus has good spatial locality)?

```
int sum_array_3d(int a[M][N][N])
{
    int i, j, k, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                sum += a[k][i][j];
    return sum;
}
```

PRINCIPLE OF LOCALITY

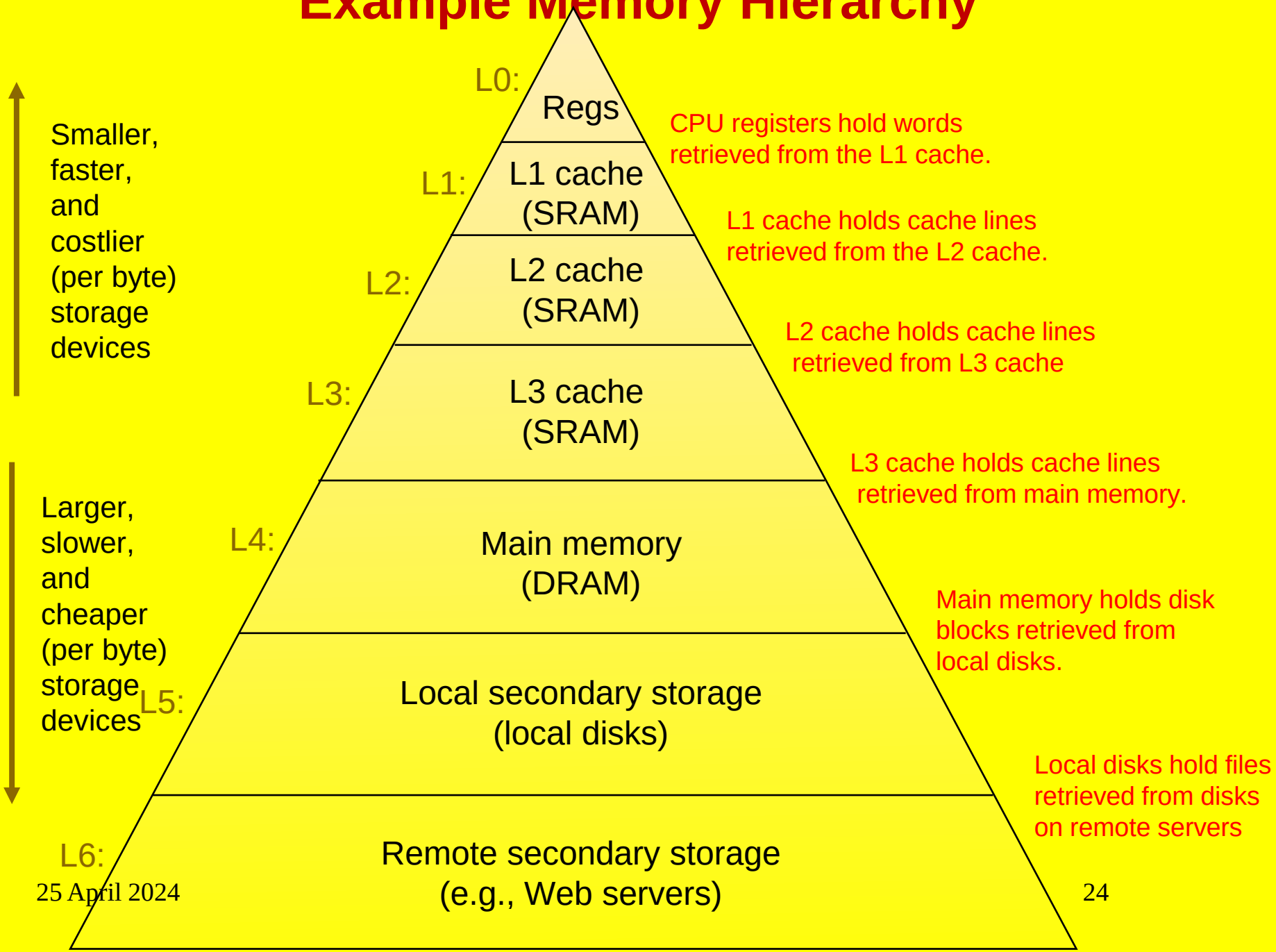
- **Some simple rules for qualitatively evaluating the locality in a program:**

- **Programs that repeatedly reference the same variables enjoy good temporal locality.**
- **For programs with stride-k reference patterns, the smaller the stride the better the spatial locality.**
- **Programs with stride-1 reference patterns have good spatial locality.**
- **Programs that hop around memory with large strides have poor spatial locality.**
- **Loops have good temporal and spatial locality with respect to instruction fetches.**
- **The smaller the loop body and the greater the number of loop iterations, the better the locality.**

Memory Hierarchies

- **Some fundamental and enduring properties of hardware and software:**
  - **Fast storage technologies cost more per byte, have less capacity, and require more power (heat!).**
  - **The gap between CPU and main memory speed is widening.**
  - **Well-written programs tend to exhibit good locality.**
- **These fundamental properties complement each other beautifully.**
- **They suggest an approach for organizing memory and storage systems known as a memory hierarchy.**

Example Memory Hierarchy



Examples of Caching in the Mem. Hierarchy

Cache Type	What is Cached?	Where is it Cached?	Latency (cycles)	Managed By
Registers	4-8 bytes words	CPU core	0	Compiler
TLB	Address translations	On-Chip TLB	0	Hardware MMU
L1 cache	64-byte blocks	On-Chip L1	4	Hardware
L2 cache	64-byte blocks	On-Chip L2	10	Hardware
Virtual Memory	4-KB pages	Main memory	100	Hardware + OS
Buffer cache	Parts of files	Main memory	100	OS
Disk cache	Disk sectors	Disk controller	100,000	Disk firmware
Network buffer cache	Parts of files	Local disk	10,000,000	NFS client
Browser cache	Web pages	Local disk	10,000,000	Web browser
Web cache	Web pages	Remote server disks	1,000,000,000	Web proxy server

Summary

- The speed gap between CPU, memory and mass storage continues to widen.
- Well-written programs exhibit a property called *locality*.
- Memory hierarchies based on *caching* close the gap by exploiting locality.

Writing Cache Friendly Code

- ***Make the common case go fast***
  - Focus on the inner loops of the core functions

- **Minimize the misses in the inner loops**
  - Repeated references to variables are good (temporal locality)
  - Stride-1 reference patterns are good (spatial locality)

Key idea: Our qualitative notion of locality is quantified through our understanding of cache memories

The Memory Mountain

- **Read throughput (read bandwidth)**
  - Number of bytes read from memory per second (MB/s)

- **Memory mountain:**

- Measured read throughput as a function of spatial and temporal locality.
  - Compact way to characterize memory system performance.

INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

25 April 2024

SOC 2040
SYSTEM PROGRAMMING

WEEK 11 LECTURE 2

MATRIX MULTIPLICATION EXAMPLE

EXPLOITING CACHE LOCALITY

Matrix Multiplication Example

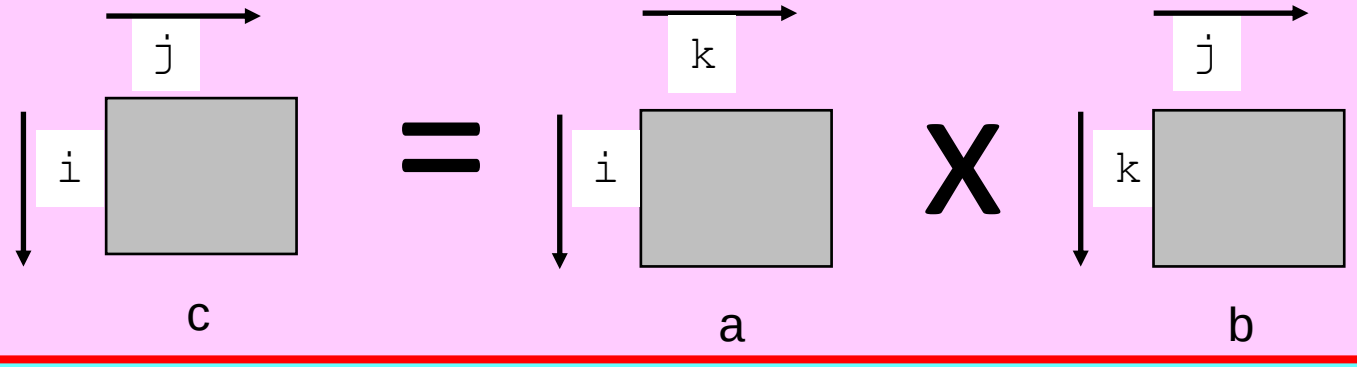
- Description:
  - Multiply N x N matrices
  - Matrix elements are double**s** (8 bytes)
  - $O(N^3)$ total operations
  - N reads per source element
  - N values summed per destination
    - but may be able to hold in register

```
/* ijk */
for (i=0; i<n; i++)  {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```

Variable `sum` *held in register*

Miss Rate Analysis for Matrix Multiply

- Assume:
  - Block size = 32B (big enough for four doubles)
  - Matrix dimension (N) is very large
    - Approximate 1/N as 0.0
  - Cache is not even big enough to hold multiple rows
- Analysis Method:
  - Look at access pattern of inner loop



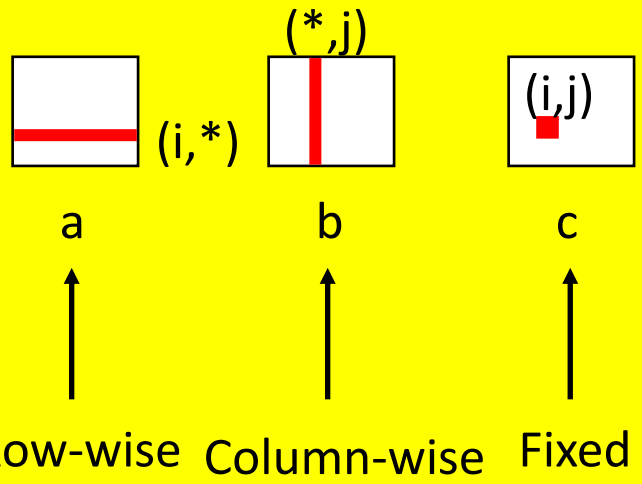
Layout of C Arrays in Memory

- C arrays allocated in row-major order
  - each row in contiguous memory locations
- **Stepping through columns in one row:**
  - `for (i = 0; i < n; i++)`
    `sum += a[0][i];`
  - accesses successive elements
  - if block size (B) > sizeof(a_{ij}) bytes, exploit spatial locality
    - miss rate = sizeof(a_{ij}) / B
- **Stepping through rows in one column:**
  - `for (i = 0; i < n; i++)`
    `sum += a[i][0];`
  - accesses distant elements
  - no spatial locality!
    - miss rate = 1 (i.e. 100%)

Matrix Multiplication (ijk)

```
/* ijk */
for (i=0; i<n; i++)  {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```

Inner loop:



Misses per inner loop iteration:

a	b	c
0.25	1.0	0.0

2 Loads & no store

Cache Block Size= 4 double elements

Elements of Matrices a, b & c are stored in Row-Major Order

Variable sum is in register

Matrix a : Row access

a[0][0] a[0][1] a[0][2] a[0][3]

Miss Hit Hit Hit

Miss Rate = ¼=0.25

Matrix b : Column access

b[0][0] b[1][0] b[2][0] b[3][0]

Miss Miss Miss Miss

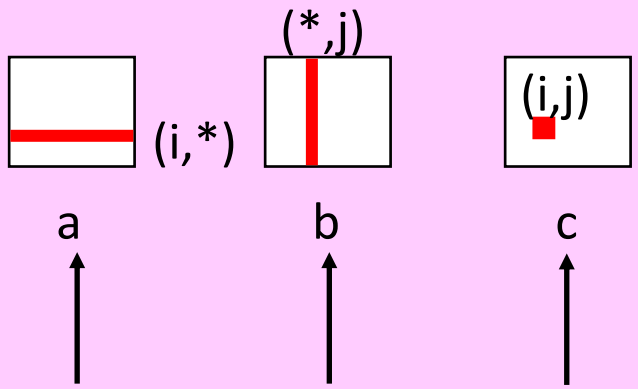
Miss Rate =4/4=1.0

Matrix c - NO ACCESS in the inner loop

Matrix Multiplication (jik)

```
/* jik */
for (j=0; j<n; j++) {
  for (i=0; i<n; i++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum
  }
}
```

Inner loop:



Misses per inner loop iteration:

a	b	c
0.25	1.0	0.0

2 Loads & no store

Cache Block Size= 4 double elements

Elements of Matrices a, b & c are stored in Row-Major Order

Variable sum is in register

Matrix a - Row access

a[0][0] a[0][1] a[0][2] a[0][3]

Miss Hit Hit Hit

Miss Rate = ¼=0.25

Matrix b - Column access

b[0][0] b[1][0] b[2][0] b[3][0]

Miss Miss Miss Miss

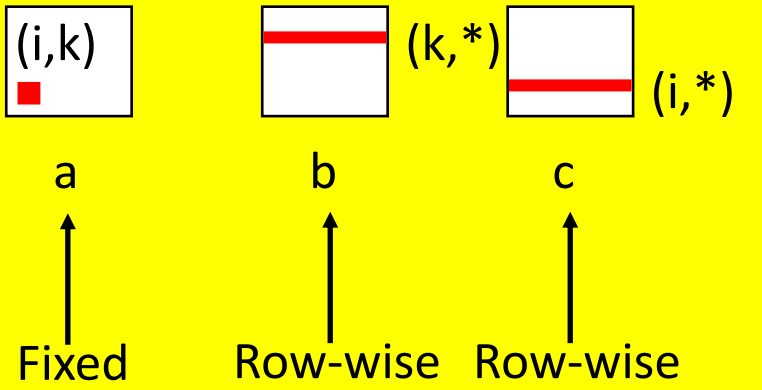
Miss Rate =4/4=1.0

Matrix c - NO ACCESS in the inner loop

Matrix Multiplication (kij)

```
/* kij */
for (k=0; k<n; k++) {
  for (i=0; i<n; i++) {
    r = a[i][k];
    for (j=0; j<n; j++)
      c[i][j] += r * b[k][j];
  }
}
```

Inner loop:



Misses per inner loop iteration:

a	b	c
0.0	0.25	0.25

2 Loads & 1 Store

Cache Block Size= 4 double elements

Elements of Matrices a, b & c are stored in Row-Major Order

Variable sum is in register

Matrix A - NO ACCESS in the inner loop

Matrix B Row access

b[0][0] b[0][1] b[0][2] b[0][3]

Miss Hit Hit Hit

Miss Rate = ¼=0.25

Matrix C Row access

c[0][0] c[0][1] c[0][2] c[0][3]

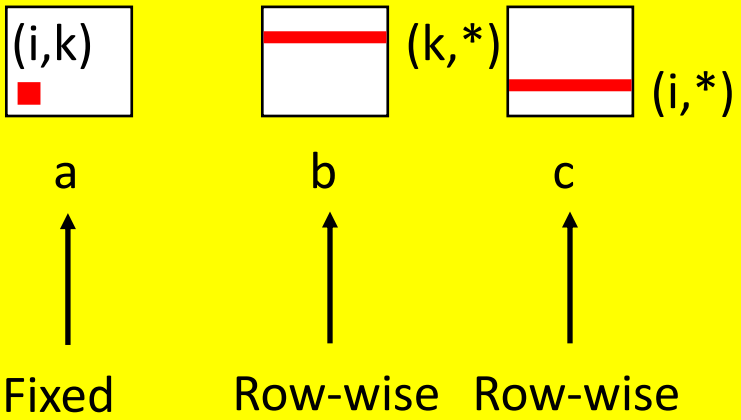
Miss Hit Hit Hit

Miss Rate = ¼=0.25

Matrix Multiplication (ikj)

```
/* ikj */
for (i=0; i<n; i++) {
  for (k=0; k<n; k++) {
    r = a[i][k];
    for (j=0; j<n; j++)
      c[i][j] += r * b[k][j];
  }
}
```

Inner loop:



Misses per inner loop iteration:

a	b	c
0.0	0.25	0.25

2 Loads & 1 Store

Cache Block Size= 4 double elements

Elements of Matrices a, b & c are stored in Row-Major Order

Variable sum is in register

Matrix a - NO ACCESS in the inner loop

Matrix b - Row access

b[0][0] b[0][1] b[0][2] b[0][3]

Miss Hit Hit Hit

Miss Rate = ¼=0.25

Matrix c - Row access

c[0][0] c[0][1] c[0][2] c[0][03

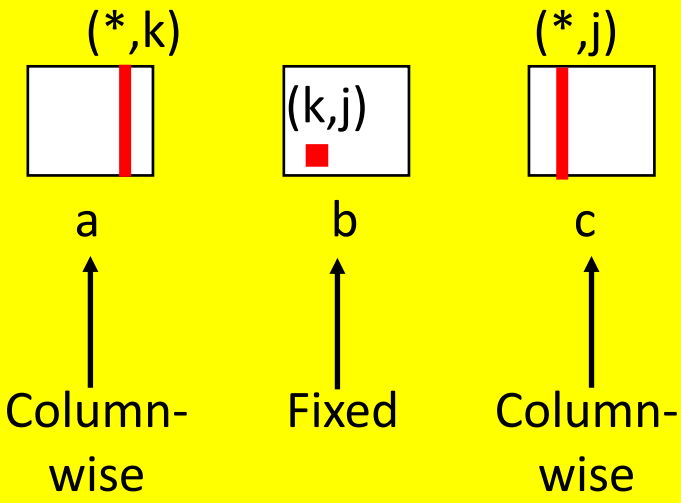
Miss Hit Hit Hit

Miss Rate = ¼=0.25

Matrix Multiplication (jki)

```
/* jki */
for (j=0; j<n; j++) {
  for (k=0; k<n; k++) {
    r = b[k][j];
    for (i=0; i<n; i++)
      c[i][j] += a[i][k] * r;
  }
}
```

Inner loop:



Misses per inner loop iteration:

a	b	c
1.0	0.0	1.0

2 Loads & 1 Store

Cache Block Size= 4 double elements

Elements of Matrices a, b & c are stored in Row-Major Order

Variable sum is in register

Matrix a – Column access

a[0][0] a[1][0] a[2][0] a[3][0]

Miss Miss Miss Miss

Miss Rate =4/4=1.0

Matrix b - NO ACCESS in the inner loop

Matrix c - Column access

c[0][0] c[1][0] c[2][0] c[3][0]

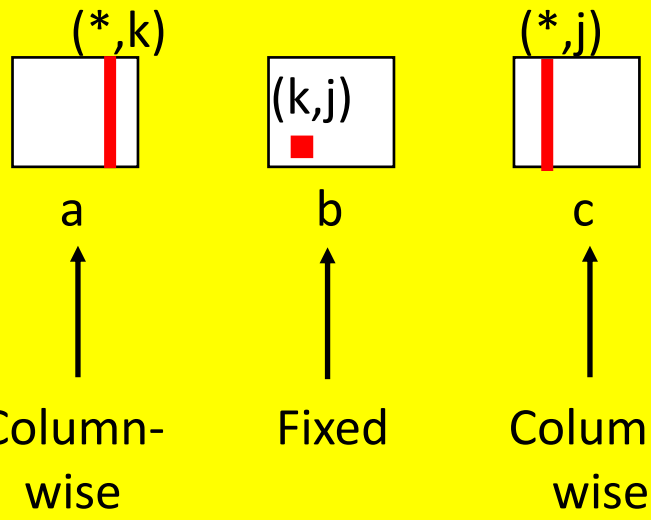
Miss Miss Miss Miss

Miss Rate =4/4=1.0

Matrix Multiplication (kji)

```
/* kji */
for (k=0; k<n; k++) {
  for (j=0; j<n; j++) {
    r = b[k][j];
    for (i=0; i<n; i++)
      c[i][j] += a[i][k] * r;
  }
}
```

Inner loop:



Misses per inner loop iteration:

a	b	c
1.0	0.0	1.0

2 Loads & 1 Store

Variable sum is in register

Matrix a – Column access

a[0][0] a[1][0] a[2][0] a[3][0]

Miss Miss Miss Miss

Miss Rate =4/4=1.0

Matrix b - NO ACCESS in the inner loop

Matrix c - Column access

c[0][0] c[1][0] c[2][0] c[3][0]

Miss Miss Miss Miss

Miss Rate =4/4=1.0

Cache Block Size= 4 double elements

Elements of Matrices a, b & c are stored in Row-Major Order

Summary of Matrix Multiplication

```
for (i=0; i<n; i++) {
  for (j=0; j<n; j++) {
   sum = 0.0;
   for (k=0; k<n; k++)
     sum += a[i][k] * b[k][j];
   c[i][j] = sum;
 }
}
```

ijk (& jik):
- 2 loads, 0 stores
- misses/iter = 1.25

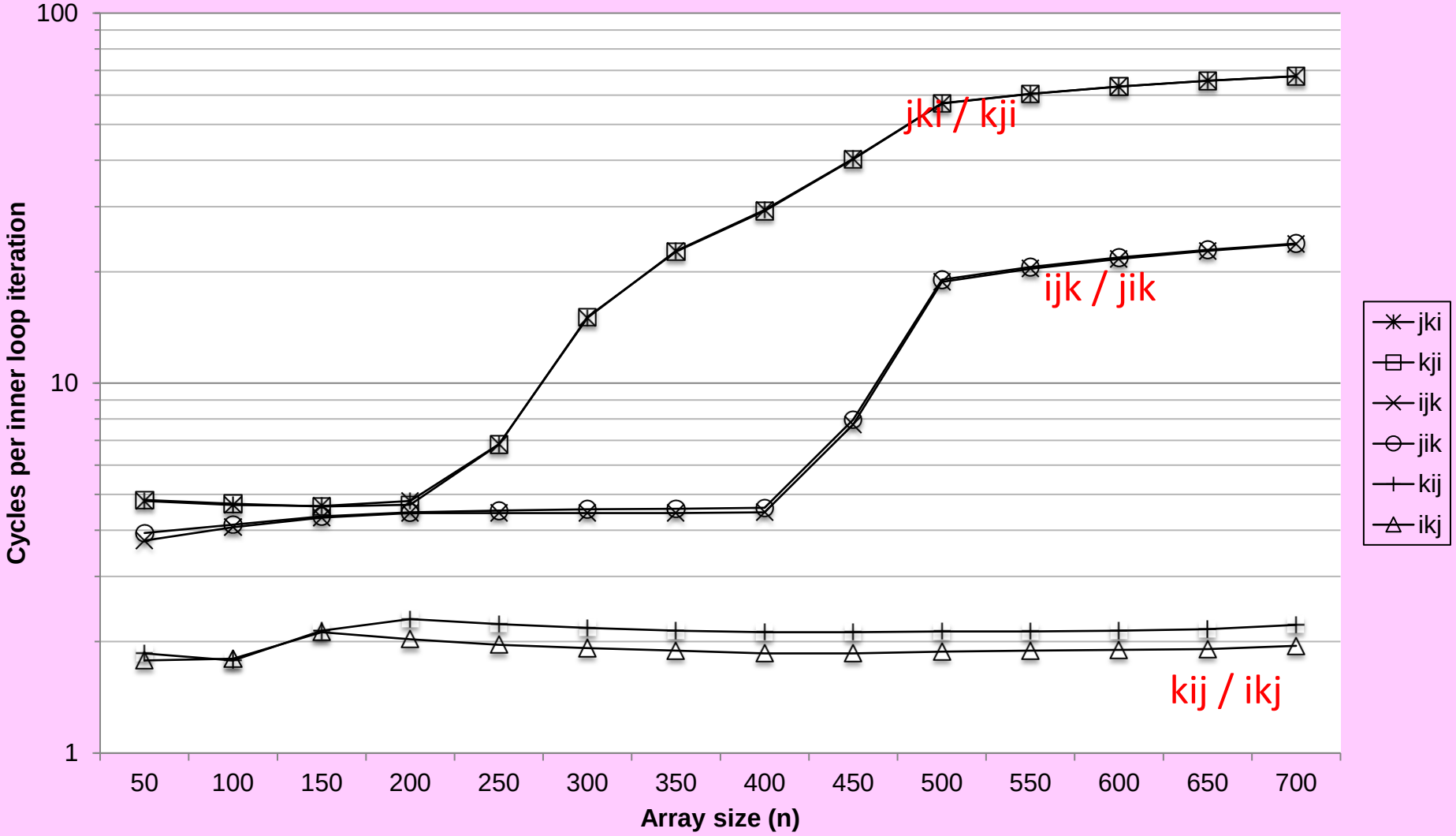
```
for (k=0; k<n; k++) {
 for (i=0; i<n; i++) {
  r = a[i][k];
  for (j=0; j<n; j++)
   c[i][j] += r * b[k][j];
 }
}
```

kij (& ikj):
- 2 loads, 1 store
- misses/iter = 0.5

```
for (j=0; j<n; j++) {
 for (k=0; k<n; k++) {
   r = b[k][j];
   for (i=0; i<n; i++)
    c[i][j] += a[i][k] * r;
 }
}
```

jki (& kji):
- 2 loads, 1 store
- misses/iter = 2.0

Core i7 Matrix Multiply Performance



Cache Summary

- Cache memories can have significant performance impact

- You can write your programs to exploit this!
  - Focus on the inner loops, where bulk of computations and memory accesses occur.
  - Try to maximize spatial locality by reading data objects with sequentially with stride 1.
  - Try to maximize temporal locality by using a data object as often as possible once it’s read from memory.

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapters 6 & 7 of Text Book

- **Computer Systems : A Programmer's Perspective**

  - **Randal E. Bryant and David R. O'Hallaron**
    3rd Edition, Global Edition Prentice Hall 2016
    ISBN 10:1-292-10176-8

SOC 2040
SYSTEM PROGRAMMING

END OF

MEMORY HIERARCHY
PART 3

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

25 April 2024